INDEX

NAME: Yuvan.s STD.: _____ SEC.: _____ ROLL NO.: 614 SUB.: _____

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1	21.7.26	Study of network commands		31/8/26
2.	27.7.26	Study of different types of network cable.	9	
3.	9/8/26	Cisco Packet Tracer	9	10/8
4.	10/8/26	Setup and configure a LAN using switch and ethernet cable in your lab	9	10/8
5.	11/8/26	Packet capture tool : Wireshark.	9	11/8
6.	17/8/26	Error correction and data link layer.	10	17/8
7.	18/8/26	Flow control and data link layer	10	18/8
8.	24/8/26	Nmap to discover live host.	10	24/8
9.	25/8/26	Implementation of subnetting	10	25/8
10.	31/8/26	Implement echo client server Implement chat client server.	9	31/8/26

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
13.	8/9/26	Implement your ping progeam.	9	8/9
14.	6/10/26	Write a code using RAW sockets in implement pocket sniffing.	9	6/10/26

Date: 21.7.26
Experiment 1

Basic Network Commands

Aim:

Study of various network commands used in Linux or windows.

1) ARP - A :

ARP - Address resolution protocol. It will show the IP address of your computer along with the IP address and MAC address of your router.

2) Hostname :

This is the simplest of all TCP/IP commands. It simply displays the name of your computer.

Output :

localhost.localdomain

3) Ipconfig/all :

This command displays detailed configuration information about your TCP/IP connection including Router, gateway, DNS, DHCP, type of ethernet adapter in your system.

4) Nbtstat - a :

This command helps solve problems with netBIOS name resolution.

5) netstat : [network statistics]

A netstat displays a variety of statistics about a computers active TCP/IP connections.
It is a command line tool for monitoring network connections both incoming and outgoing as well as routing tables, interface statistics etc.

6) nslookup : [name solver lookup]
is a tool used to perform DNS lookups in linux. Used to display DNS details, such as the IP address of the particular computer. The MX records for a domain or the NS servers of a domain. Ns lookup can operate in two modes: interactive & non-interactive.

7) Pathping :
It is unique to windows & is basically a combination of the ping & the tracert commands.

8) Ping :
This command is to test connectivity between two nodes. It uses ICMP to communicate to other devices.

9) Route:

Route command is used to show manipulate the IP routing table. It is primarily used to setup static routes to specific host on network via an interface.

LINUX COMMANDS:

1) IP: This command is one of the basic commands which every administrator needs in daily work from setting up new system & assignments IPs to troubleshooting, manipulate routing, plus display network various devices interface & tunnels.

2) ifconfig:

This command is staple in many sys admins tools belt for configuring & troubleshooting networks. It has been replaced by the Ip commands discussed above.

3) mtr: matts trace route, is a program with a command line interface that serves as a network diagnostic & trouble shoot tool.

This command combines the functionality of the ping & traces out commands.

4) tcpdump:

tcpdump is a command designed for capturing & displaying packets.

5) ping: It is a tool that verifies IP level to another TCP/IP computer by sending internet control message protocol (ICMP) echo request message. Ping is the primary TCP/IP command used to troubleshoot connectivity.

Result:

Thus the various network commands used in Linux & windows are studied & the output is verified.

Study of Network commands

Window Commands

1) ARP - A : ARP is short form of address resolution protocol.

O/p : Interface : 10.16.56.50 --- 0x9

Internet address	Physical address	Type
10.16.56.155	ec-12-17-f7-f2-5c	dynamic
10.16.56.255	ff-ff-ff-ff-ff-ff	static

2) hostname : This is the simplest of all TCP/IP commands. It will display the name of your computer.

O/p : LAPTOP 7003BKTN

3) Ipconfig/all : This command displays configuration information about your TCP/IP connection.

O/p : Window IP configuration

Host name Laptop-7003BKTN

Node type .....

4) nbtstat -a : This command helps solve problems with netBIOS name resolution.

o/p : Ethernet 2

Node Ip Address : [192.168.56.

Scope Id : [ ]

Host not found

5) nb netstat : (network statics)
It displays a variety of statistics about a computer active TCP/IP connection.

o/p : Active connection

Proto	local address	Foreign address	static
Tcp	10.16.56.50.4	9444 4.213.25.242: https	Establishe

6) Server : Unknown
Address : 10.16.56.155

nslookup : Used to display the dns detail, such as Ip address of a particular computer.

o/p :

7) Pathping : is a unique to windows and basically a combination of the ping tracert commands.

o/p: tracing route to google.com
[142.250.66.14] over a
max of 30 hops.
Trace complete.

8) Ping: this command is the bat way to tat connectivity between 2 nodes.

Ping.google.com

o/p:
Pinging.

9) Route: route command is used to show/manipulate the IP routing table. primarily used to setup static routes on networ via a interface.

Manipulate network routing tables
Route [-f] [-p] [-4] [-6]
command declaration.

Linux Commands

1) IP commands :-
-> cmd: displays all IP ad
of network interface.

o/p: etho : 192.168:1.10/2

→ ip address add 192.168.1.254/24
an ip address to an interface

o/p: Ip address added successfully

3) ip link etho up : Enables the network interface

o/p: etho state up.

4) ip link set etho down -
disables the network interface

o/p: etho state down

5) ip link set etho promise on -
enables promiscuous mods.

o/p: promiscuous mode enabled.

6) Ip route - add default via 192.168.1.254 dw → adds default gateway.

7) ip route get 10.10.1.4 : Shows the route to a destination Ip.

o/p: 10.10.1.4 via 192.168.1.254 dw etho

2) ~~ifco~~ ifconfig : displays network interface configuration.

o/p : etho link encap : Ethernet
inet 192.168.1.10

3) mtr

1. mtr google.com - shows route statics to a host.

o/p :- HOST : localhost

2. mtr -g google.com - shows numeric Ip address.

o/p : 142.250.183.286

3. mtr -c 10 google.com - Shows both hostname & Ip address.

o/p : google.com (142.250.183.206)

4. mtr -c 10 google.com - sends 10 probe packets

o/p : sent 10 packets

4) tcpdump :-

1) dnf install - tcpdump

2) tcpdump -p - cat available capture interface

o/p : 1: etho 2: Io

3) tcpdump -i etho -c 10 - capture packets to 1 from a host.

o/p : 10 packets captured

4. tcpdump -i eth0 host 8.8.8.8 - captures packets from a host

o/p : 8.8.8.8 > 192.168.1.10

5. tcpdump -i eth0 host 8.8.8.8 captures packets to a host

6. tcpdump -i eth0 port 53 - captures DNS traffic

o/p : DNS captured

7. tcpdump -i eth0 -c 10 host www.google.com 4 443 - captures HTTPS packets

o/p : TCP, 443 captured

5) Ping :

1) Ping google.com - checks connectivity to a host

o/p : 64 bytes from google.com

2) Ping -c 10 google.com sends only to ping request

o/p : 10 packets transmitted
-10 recorded

Result :

Thus the various network commands used in linux & windows are studied & the output is verified.

27.7.20
Exp. 2

Study of different types of network cable

Aim:

To study different types of network cable.

a) Understand different types of network cable.

1) Unshielded twisted pair (UTP) cable.
2) Shielded twisted pair (STP) cable.
3) Coaxial cable.
4) Fibre optic cable.

Cable type	Category	Maximum data transmission	Advantages / Disadvantages	Application / Use	Image
UTP	Category 3	10 bps	Advantage: → Cheaper in cost → Easy to install as they have a smaller overall diameter. Disadvantage: → More prone to (EMI) electromagnetic interference & noise.	10 Base - T ethernet	
	Category 5	Up to 100 mbps		Fast ethernet, gigabit ethernet	
	Category 5e	1 Gbps		Fast ethernet, gigabit ethernet	
STP	Category 6, 6a	10 Gbps	Advantage: → Shielded → faster than UTP → Less susceptible to noise & interference. Disadvantage: → Expensive. → Greater installation effort.	Gigabit ethernet, 10 G ethernet (55 m) data centre.	
SSTP	Category 7	10 Gbps		Gigabit ethernet, 10 G ethernet (100 m)	

Co-axial cable	RG-6 RG-59 RG-11	10-100 Mbps	Advantage: → High bandwidth → Immune to interface → Low loss bandwidth → Versatile Disadvantage: → Limited distance → Cost → Size is bulky	Speed of signal is 500m television network. High speed internet connections.
Fibre optics cable	Single mode multi mode	100 Gbps	Advantage: → High speed → High bandwidth → High security → Long distance Disadvantage: → Expensive → Requires skilled installers	Maximum distance of fibre optics cable is around 100 meters.

Result:

Thus the study of different types of network cable studied successfully.

27/7/20

Cisco Packet Tracer

Aim:

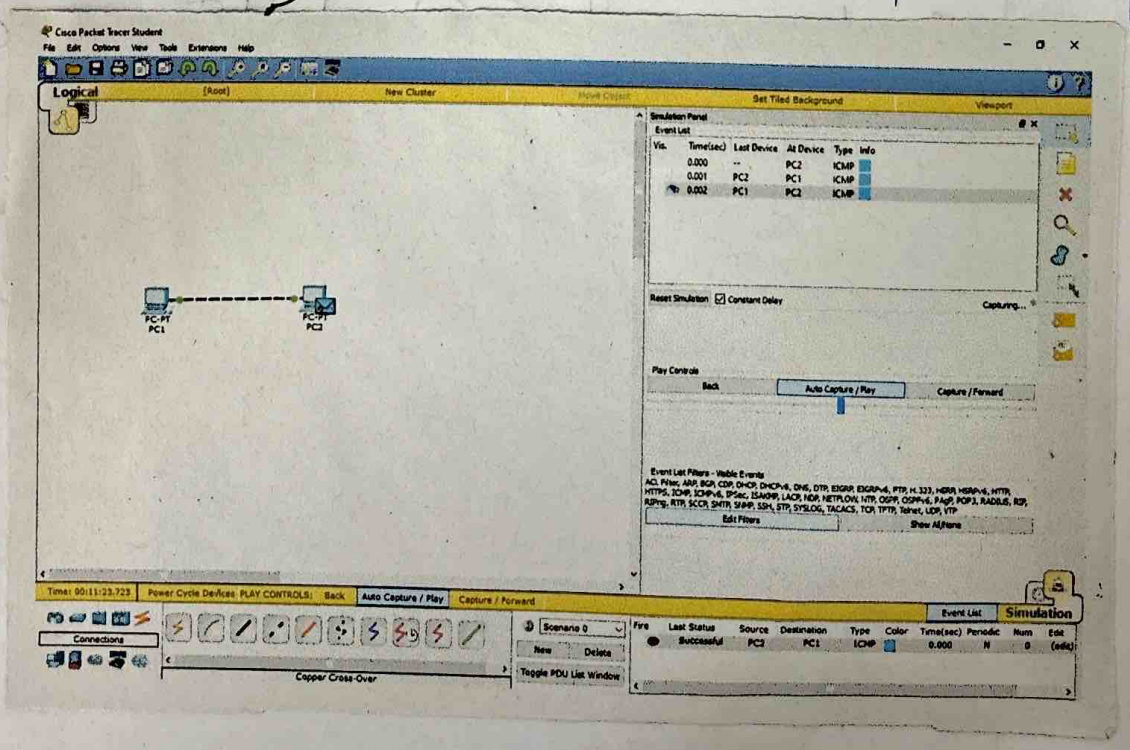
To study the cisco packet tracer simulation tool, understand its user interface, design a simple network & analyze the behaviours of hub, switch & peer to peer.

Cisco packet tracer:

cisco packet tracer is a network Simulation & modeling tool. It allows to create & test computer network without using real networking equipment we can add PCs switches hubs, routers & other devices & observe how packets travel between them.

① Peer to Peer

Two computer communicate directly



→ Place two PCs & connect them using copper cross over cable.

→ Configure IP address for both PC's in the same network.

→ Use ping to verify the connection.

② Hub Network:

→

→ Place the PC's & a Hub, the connect them using copper straight through cable.

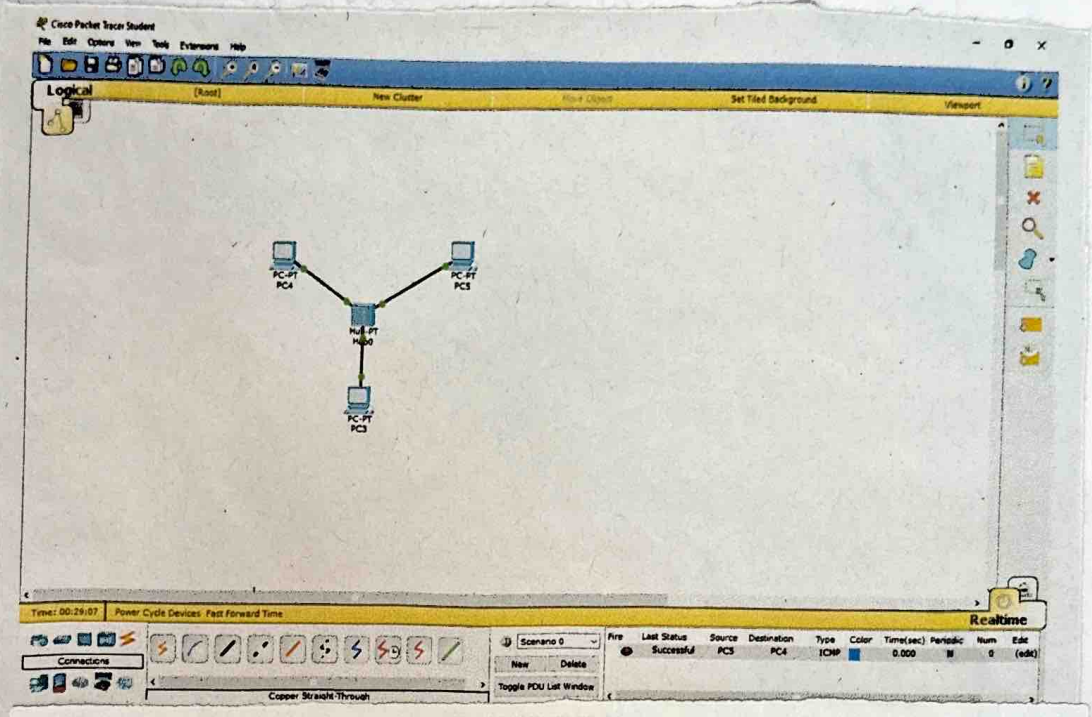
→ Use ping to verify connection & the communication in PC's.

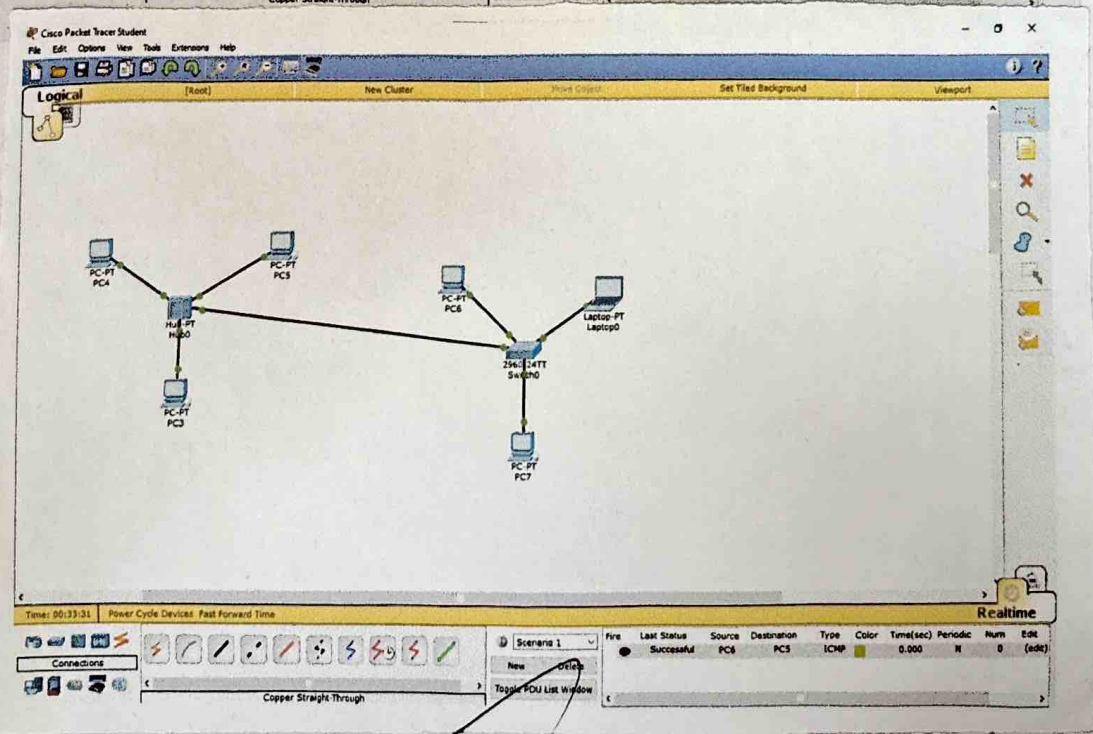
③ Switch Network:

→

IP configuration:

Device	IP address	Subnet Mask
PC0	10.1.1.1	255.00.0
PC1	10.1.1.2	255.0.0.0





Result:

Cisco packet tracer was successfully studied & used to create a simple network.

8/8/26

9

Setup & configure LAN using a switch & ethernet cable in your lab.

Aim: To setup & configure a LAN using a switch & ethernet cable in cisco packet tracer & verify the connectivity between the devices.

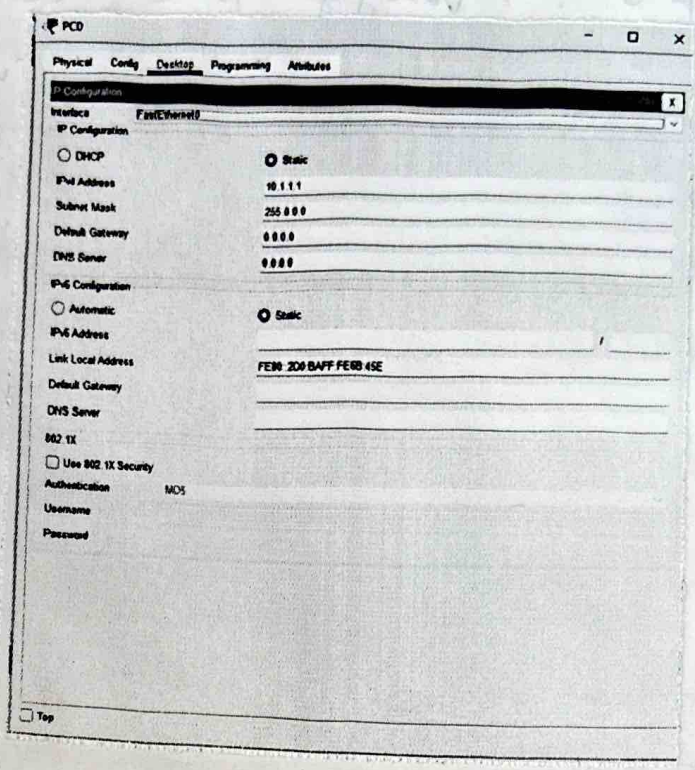
Procedure:

Step 1: Open cisco packet tracer and create a LAN topology using 4 PCs & 1 switch.

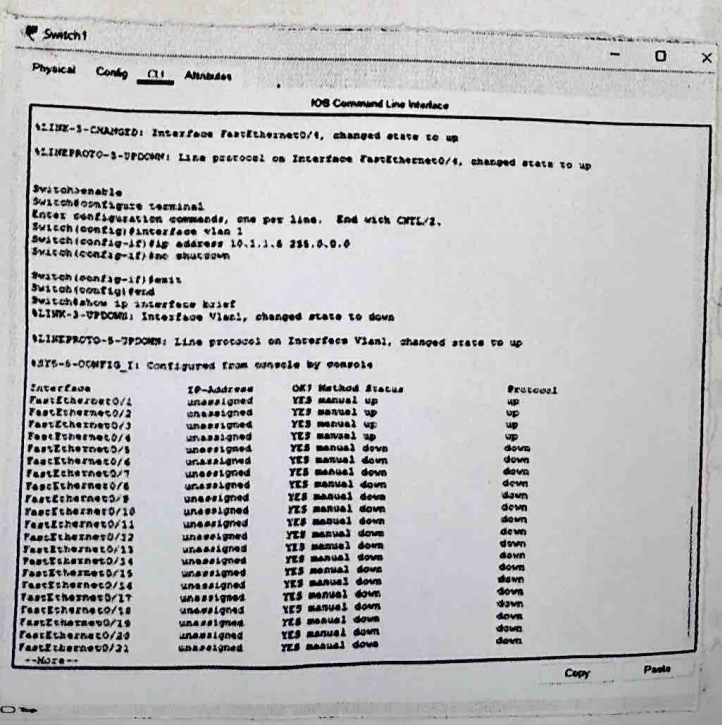
Step 2: Connect all the PCs to the switch using copper straight - through ethernet cable.

Step 3: Configure the IP address for each PC using the following setting.

Device	IP address	Subnet mask



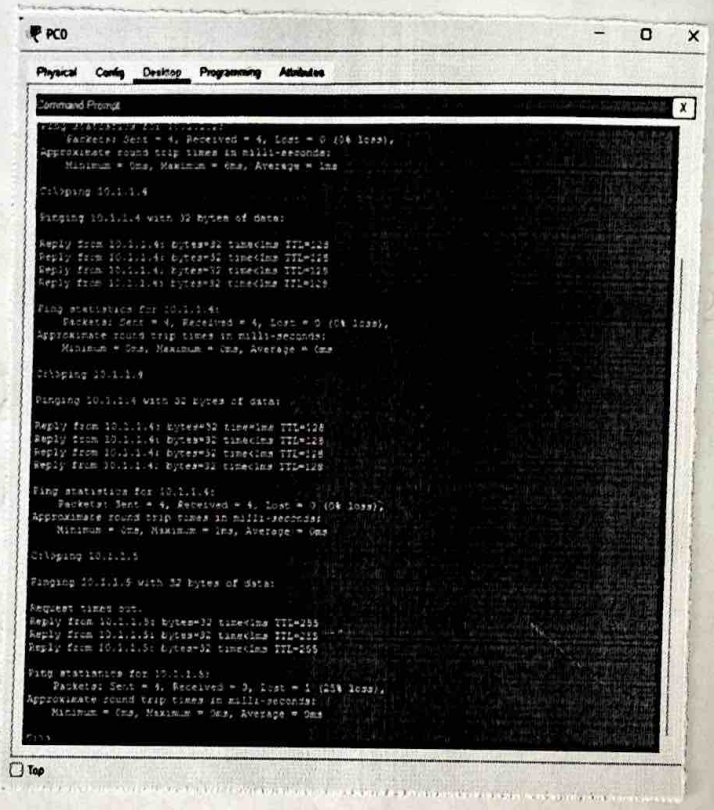
Step 4: Configure the width using the CLI. Assign the switch SVI 1 interface the IP address 10.1.1.5 with subnet mask 255.0.0.0 & enable the interface using the no shutdown command.



Step 5: Verify the connectivity between the PCs & switch using the ping command from the command prompt.

eg. Ping 10.1.1.4

Ping 10.1.1.5



Result: Thus, a LAN was successfully configured using a switch & ethernet cable in cisco packet, and the connectivity between the devices was successfully verified ping command.

10/12

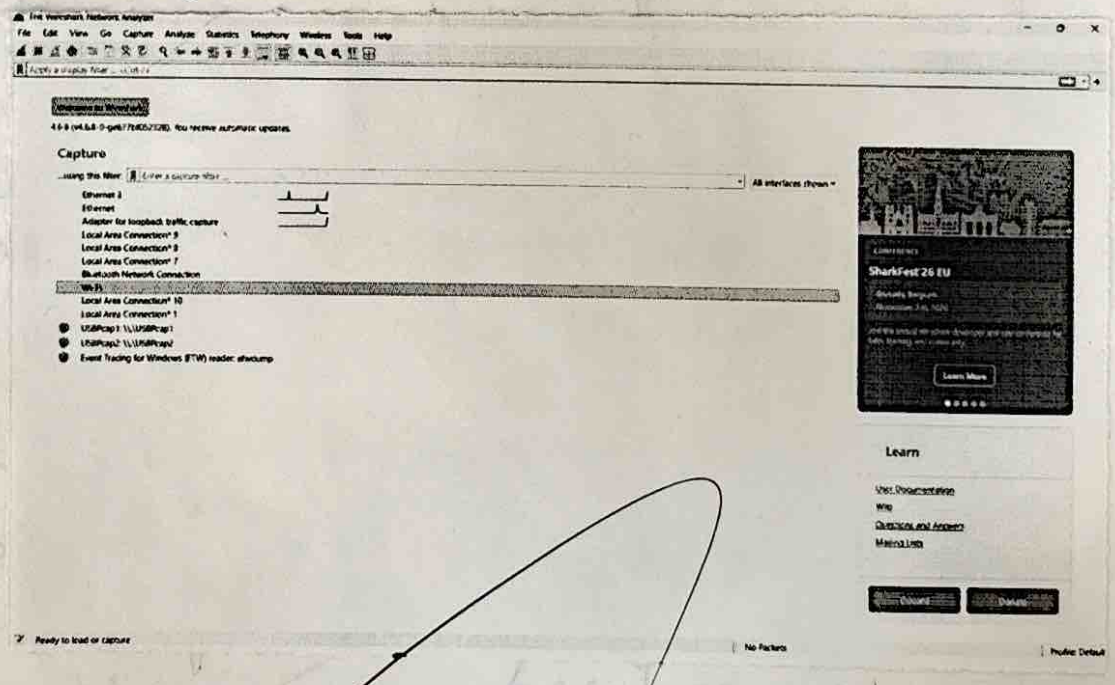
Packet capture tool: Wireshark

Aim:

To understand the features of wireshark as a packet capture tool & understand encapsulation of information at various layers of a protocol.

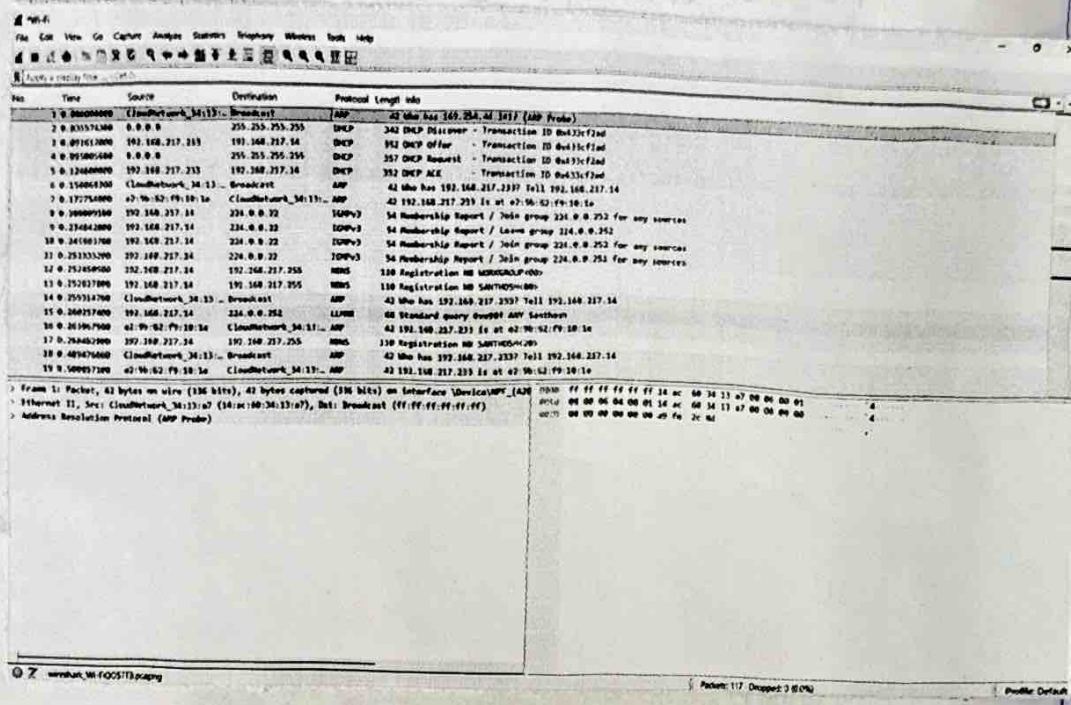
Procedure: Step 1: Open wireshark & select the wifi network interface.

Step 2: Start the packet capture by double clicking the wifi interface.



Step 3: Observe the packet being captured in the wireshark window.

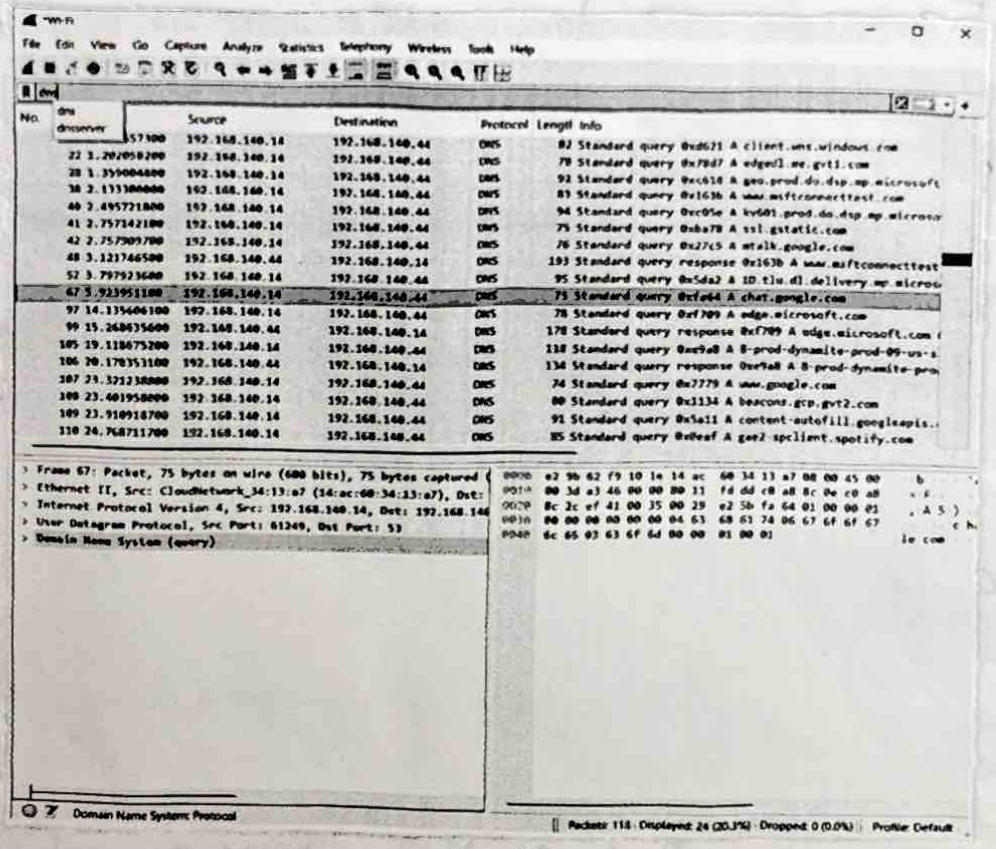
Step 4: Different packet such a ARP, DHCP, NDNS & MVR can be observed in the packet list.



Step 5: Select any packet to view its detailed information such as frame, ethernet II, IP UDP/TCP & protocol details.

Step 6: Apply the display filter due in the filter bar to display only DNS packet.

Step 7: Observe DNS query & respond The source & IP address, protocol, packet. Length & DNS information on be analyzed.



Step 8: Stop the packet capture after collecting the required packet.

Result:

Network packet was successfully captured & analyzed using wireshark & DNS packet were identified using the display filter.

12/8/26

Exp.6 Error correction at data link layer.

Aim: Write a program to implement error correction using hamming code concept. To input data stream & verify error correction feature.

Code:

```
data = input ("Enter 4 data bits")
d1, d2, d3, d4 = map (int, data)
p1 = d1 ^ d2 ^ d4
p2 = d1 ^ d3 ^ d4
p4 = d2 ^ d3 ^ d4
code = [p1, p2, d1, p4, d2, d3, d4]
print ("Hamming code :", join (map (str, code)))
recvd = list (map (int, input ("Enter received code")))
s1 = recvd [0] ^ recvd [2] ^ recvd [4] ^ recvd [6].
s2 = recvd [1] ^ recvd [2] ^ recvd [5] ^ recvd [6].
s4 = recvd [3] ^ recvd [4] ^ recvd [5] ^ recvd [6]
error = s4 * 4 + s2 * 2 + s1
if error == 0:
    print ("No e
else:
    print ("Error at
    received [error - 1
    print ("Corrected
(str, received)))
```

o/p:

```
Enter 4 data bits
Hamming code : 00
Enter received cod
Error at positio
Corrected code :
```

Result:

~~Thus~~, the Ha
generated, the
the bit

```
if error == 0:
    print ("No error")
else:
    print ("Error at position:", error)
    received [error - 1] ^= 1
    print ("Corrected code:", "".join (map
(str, received)))
```

o/p:

Enter 4 data bits : 1001

Hamming code : 0011001

Enter recived code : 1001011

Error at position : 4

Corrected code : 1000011.

Result :

Thus, the Hamming code was successfully generated, the error was detected and the bit was correctly placed.

17/8/25

Ex. 7 Flow control at Data Link Layer

Aim:
Write a program to implement flow control at data link layer using sliding window protocol. Simulate the flow of frames from one node to another.

Code:

Sender.py

```
message = input("Enter message: ")
window = int(input("Enter window size: "))
i=0
while i < len(message):
    print("In Sender Buffer:")
    for j in range(i, min(i+window, len(message))):
        print("Wrong ACK - resending frames")
print("In all frames sent successfully");
```

receiver.py

```
frame = int(input("Enter received frame number: "))
        data = input("Enter frame data: ")
        expected = int(input("Enter expected frame number: "))
        print("\n receiver: ")
        frame = expected;
        print("Frame", frame, "received")
        print("Data", data)
        print("Ack", frame, sent")
    else:
        print("wrong frame")
        print(" Ack sent")
```

o/p:

Sender:

Enter message: hello
Enter window size: 3
Sender button
frame 1: h
frame 2: e
frame 3: l

Enter Ack received: 5

Frame 1: h
Frame 2: e
Frame 3: l

Enter Ack received : 3

Ack received & accepted

Ack accepted

Sender buffer

frame 4 : 1

frame 5 : 0

All frame receiving.

Receiver:

Enter received frame no: 1
enter frame data
enter expected frame no. 1

receiver:

frame 1 received
Data
Ack 1

Result

~~Received~~. Thus the sliding window protocol was successfully implemented.

18/8/26

Ex.8 NMAP to discover life host using NMap Scans (ARP, TCRP; TCP/UDP on the tryhackme platform.

Aim: To discover live host using NMap scans, (ARP, TCP/UDP) on the tryhackme platform.

Procedure:

① Setup & environment initiation
Log in to the tryhackme platform & navigate to the target room: Tryhackme Nmap room.
Deploy the target virtual machine and establish a VPN connection using open VPN on launch the browser based attack box.

② ARP scan for local host discovery
Perform an ARP request scan to identify active hosts on the local network segment
Command:
Sudo nmap PR -sn < target-IP (subnet)

3) ICMP ping scan:

Send ICMP Echo request packets, check for ICMP Echo reply from active hosts.

Command:

nmap - PE - sn < target - IP/subnet>

4) ICMP timestamp ping scan:

Send ICMP Timestamp request packet to discover host that block standard ICMP Eho request.

and:

sudo nmap - pp - sn < target - IP/subnet>

5) ICMP address ping scan:

Send ICMP address request packet to identify live systems by requesting subnet mask information.

6) TCP SYN ping scan:

Send TCP SYN packets (typically to post) to discover systems based on SYN - ACK (or) RST responses

and:

sudo nmap - ps -sn < target - IP/ subnet >

7) TCP ACK ping scan:

Send TCP Ack packet RST responses from active rules

and:

sudo nmap - pA - sn < tar

8) UDP ping scan:

Send empty UDP pa destination ports to dete ICMP port unreachable

and:

sudo nmap - pu - An <

9) Disable port scanning only)

use the sn flag w perform host discovery without triggering subseq

and:

sudo nmap -sn < tar

Result:

Thus, live host disc Scanning techniques (A timestamp, TCP SYN/ACK Scans) was success full the tryhackme platform

24/8/24

7) TCP Ack ping scan:

Send TCP Ack packets to dict RST responses from active hosts, bypass rules.

and:

sudo nmap -pA -sn <target-IP/subnet>

8) UDP ping scan:

Send empty UDP packets to destination ports to detects host via ICMP port unreachable response.

and:

sudo nmap -pu -Ath <target-IP/subnet>

9) Disable port scanning (Host discovery only)

use the sn flag with nmap to perform host discovery exclusively without triggering subsequent port scans.

and:

sudo nmap -sn <target-IP/subnet>

Result:

Thus, live host discovery using nmap Scanning techniques (ARP, ICMP and timestamp, TCP SYN/Ack, and UDP ping scans) was successfully performed on the tryhackme platform.

24/8/24

in cisco packet tracer simulate.

Aim:
To design, configure & simulate a subnetted network topology in cisco packet assign static IP address using IP subnetting multiple subnets & observe packet transmission.

Procedure:

① Design topology: Open cisco packet tracer & place routers, switches & pcs & on the workspace.

② Connect cables: Connect the pcs to switches & switches to routers interface using straight through cable.

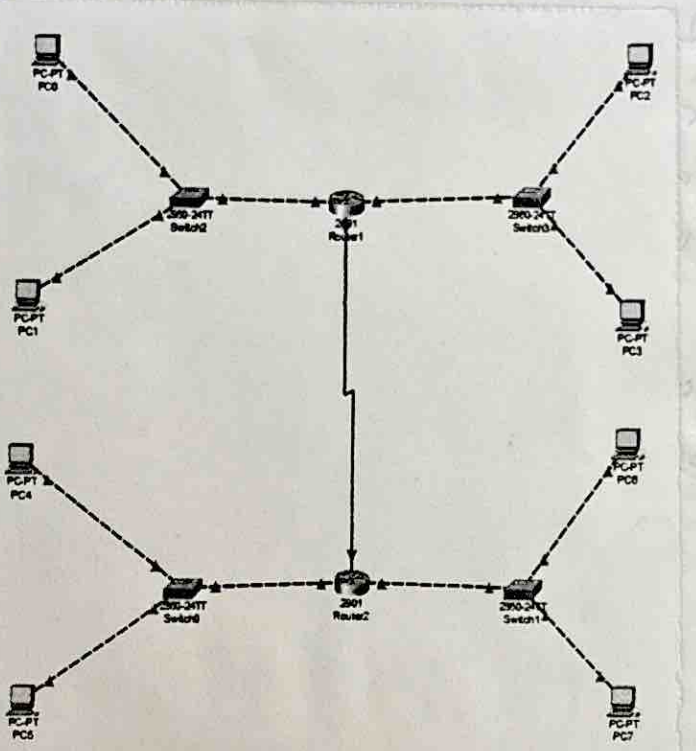
③ Assign IP address to PCs, go to desktop IP configuration each PC & assign

Subnet mask: 255.255.255.224 (127)

Subnet 1: PC IPs = 192.168.1.11 to 192.168.1.15 | default gateway = 192.168.1.1

Subnet 2: PC IPs = 192.168.2.11 to 192.168.2.15 (default gateway: 192.168.2.1)

4) Configure router: Open router CLI & configure interface)



5) Test communication:

With router: ping a PC in subnet 2 from subnet 1 using command prompt (ping 192.168.2.11) ping will be successful.

Without router: disconnect the router ping again.

Result: The subnetted network topology was successfully designed & configured in cisco packet tracer.

28/8/26

Ex. 12 Implement echo client server using TCP/UDP Socket

Aim:
To write the program to implement client server using TCP/UDP sockets.

Code:

Server.py

```
import socket
S = socket.
S.bind (('127.0.0.1',
          6543 2 ))
print ("Server ready...")
whn, addn = S.accept()
msg = conn.receive (1024)
print ("Received: " msg.
          decode())
conn.send (msg)
conn.close()
S.close()
```

Client.py

```
import socket
S = socket.socket()
S.connect (('127.0.0.1',
          6543 2 ))
S.send (b"Hello Server")
reply = S.recv (1024)
print ("Echo from server"
reply.decode())
S.close()
```

Result:

Thus, the above program is executed successfully.

31/8/24

B) Implement chat client server using TCP/UDP socket.

Aim: To write the program to implement the chat client server using TCP/UDP sockets.

Code:

Chat_Server.py

```
import socket
S = socket.select()
S.blind(('127.0.0.1', 65432))
S.listen(1)
Print("Waiting for client ...")
Conn, addr = S.accept()
while true:
    msg = conn.rev(1024).
          decode()
    if msg.lower() == 'bye';
    break;
    print("Client: ", msg)
    reply = input("Server: ")
    conn.send(reply.encode()
    if reply.lower() == 'bye';
        break;
    Conn.close()
    S.close()
```

chat. client py

```
import socket
s = socket.socket()
s.connect(('127.0.0.1',
           65432))
while True:
    msg = input("client:")
    s.send(msg.encode())
    if msg.lower() == 'bye':
        break
    print("Server:", reply)
s.close()
```

Result:

Thus the above program is executed successfully.

31/8/24

Ex.13 Implement your own pins program

Aim:

To write the program to implement your own ping program.

Code:

Server.py

```
import socket
server = socket.socket (
Socket, AF_INF, socket.sock_
DGRAM)
Server.bind (("127.0.0.1",
              12000))
print ("Server is listening...")
while true:
    data, address =
      server.recvfrom (1024)
    print ("Received:", data.
                decode())
    Server.sending (data,
      address)
```

Client.py

```
import socket
import time
client = socket.socket
(socket.AR_INET,
```

```
socket.SOCK)
client.settimeout(1)

for i in range(1,6):
    start = time.time()
    client.sendto(f"ping{i}",
    encode(),
      ("127.0.0.1",12000))

try:
    data, _ = client.recv(msgfrom(1024)
    rtt = (time.time() - start)*1000
    print(f"reply: {data.decode()} time
          = {rtt:.2f} ms")
    except socket.timeout:
    Print("Request timed out")
     client.close()
```

Result:

The above code is executed successfully.

8/9/24

Ex. 14 Write a code using RAW sockets to implement packet sniffing.

Aim:
To write a code using RAW socket to implement

Code:

Sniffer.py

```
import socket
import struct
that = socket.gethostbyname(
    Socket.gethostbyname())
sock = socket.socket(socket.
    AF_INET, socket.
sock - RAW, socket.IP)
sock.bind((HOST,0))
IP_HDR, 1)
    sock.ioctl(socket_SIO_R(
        VALL, Socket)
    Print("Packet Sniffing ...")
    Print("Host IP:", HOST)
    Print("Press ctrl+C to stop
        \n")
try:
    while True:
        Packet, address = sock(65435)
```

```
ip_header = packet[:20]
iph = struct.unpack("!BBHHHBBH4s4s" , ip_header)

source_ip = socket.inet_ntoa(iph[8])
destination_ip = socket.inet_ntoa(iph[9])

protocol = iph[6]
print("......")
print("Source IP: ", source_ip)
print("destination IP: ", destination_ip)
print("Protocol : ", protocol)
print("Packet size : ", len(packet))
print("...")

Except keyboard Interrupt:
    print("\n sniffing stopped")

finally:
    lock.ioctl(sock.SIO_RCVALL, socket.RCVALL_OFF)
    sock.close().
```

Result:

The above code is executed successfully.

6/10/24